Simulateur de modèles de données NoSQL

Big Data Structure — Homework 2.7 : explication du code, de la logique et des choix d'architecture

1. Objectif

La section 2.7 du sujet demande un **package** qui automatise les calculs des sections 2.5 (tailles) et 2.6 (sharding). Concrètement, le programme doit :

- prendre en entrée un **JSON Schema** et un **jeu de statistiques** ;
- construire une **structure (objet)** réutilisable par les programmes des séances suivantes (opérateurs de filtre, de jointure, d'agrégation) ;
- calculer la taille des **documents**, des **collections** et de la **base** ;
- calculer la **distribution des documents** d'une collection pour une clé de sharding donnée.

Le package est testé sur les cinq dénormalisations DB1 à DB5 de la section 2.4.

2. Architecture

2.1 Arborescence

Fichier	Rôle
`bigdata_sim/__init__.py`	Une ligne : expose les classes et `load_database` pour pouvoir écrire `from bigdata_sim import load_database`.
`bigdata_sim/model.py`	Toute la logique : constantes de taille, classes `Field` / `Collection` / `Database`, chargement du JSON Schema, calculs de taille et de sharding.
`schema/db1.json … db5.json`	Les JSON Schemas des cinq dénormalisations (données d'entrée).
`stats.json`	Les statistiques de la section 2.2 (données d'entrée).
`main.py`	Script de démonstration : affiche les réponses 2.5 et 2.6 pour DB1 à DB5.
`test_bigdata_sim.py`	Tests unitaires avec des valeurs calculées à la main.
`README.md`	Mode d'emploi, conventions et formules.

2.2 Pourquoi cette architecture

- **Un seul fichier de logique.** Le code fait environ 150 lignes : le découper en six modules (chargement, stats, tailles, sharding…) aurait obligé à naviguer entre fichiers sans rien apporter. Les sections du fichier jouent ce rôle de séparation.
- **Un dossier package plutôt qu'un script.** Le sujet demande un package qui sera « branché » avec les suivants. Les opérateurs du TD 3.3 et 4.2 pourront être ajoutés dans un fichier voisin (ex. `bigdata_sim/operators.py`) qui importera `Database`.
- **Données séparées du code.** Les schémas et les statistiques sont dans des fichiers JSON. Pour le challenge final (nouveau schéma, nouvelles stats), il suffit de changer ces fichiers : aucune valeur

métier (nombre de produits, de serveurs…) n'est codée en dur dans `model.py`.

- **Les calculs sont des méthodes de Database.** Une taille de document dépend à la fois du schéma (quels champs) et des statistiques (combien d'éléments dans un tableau). L'objet `Database` détient les deux, c'est donc lui qui calcule.
- **Aucune dépendance externe.** Uniquement la bibliothèque standard (`json`, `unittest`). Le projet tourne avec un Python nu.

2.3 Flux d'exécution

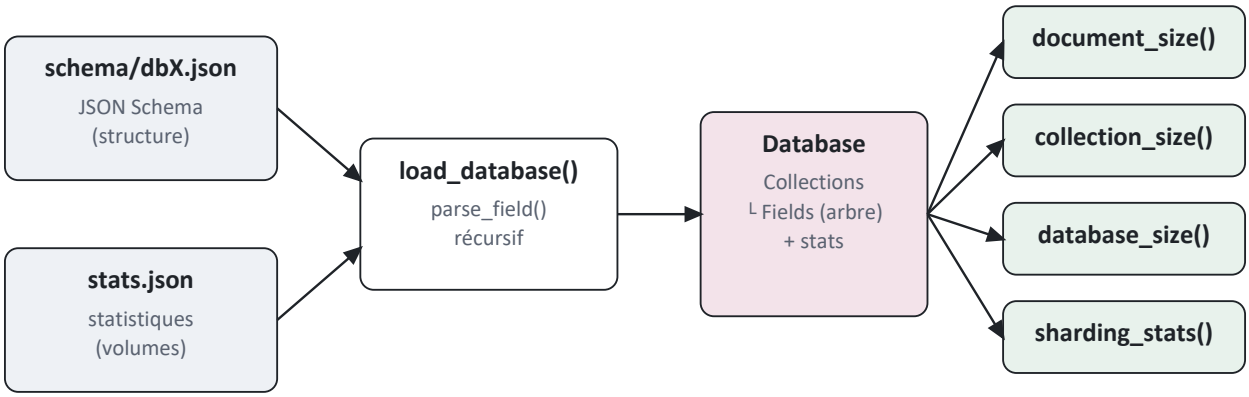


Figure 1 — Flux : entrées JSON → objet Database → fonctions de calcul

Le chargement transforme une fois pour toutes le JSON Schema en un arbre d'objets Python. Ensuite, toutes les fonctions de calcul parcourent cet arbre ; elles ne relisent jamais les fichiers.

3. Les entrées

3.1 Le JSON Schema

Chaque fichier `schema/dbX.json` est un JSON Schema (draft-07). Conventions retenues :

- chaque propriété à la racine est une **collection** de la base ;
- chaque objet liste ses clés dans `required` (le sujet impose que toutes les clés soient obligatoires) ;
- un objet imbriqué est un `"type": "object"`, un tableau imbriqué un `"type": "array"` avec `items` ;
- JSON Schema ne distingue pas les tailles de chaînes ; on utilise donc le mot-clé standard `format` : `"date"` pour une date, `"longstring"` pour un texte long (description, commentaire).

Extrait de DB2 (Product avec le tableau de stocks imbriqué) :

```
"Product": {
  "type": "object",
  "properties": {
    "IDP":         { "type": "integer" },
    "description": { "type": "string", "format": "longstring" },
    ...
    "Stock": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "IDW":      { "type": "integer" },
          "quantity": { "type": "integer" },
          "location": { "type": "string" }
        },
```

```
        "required": ["IDW", "quantity", "location"]
      }
    }
  },
  "required": ["IDP", "name", "price", "brand", "description", "image_url",
               "Categories", "Supplier", "Stock"]
}
```

3.2 Les statistiques

Clé de stats.json	Contenu	Utilisée pour
`servers`	1 000 serveurs	Sharding
`nb_docs`	Nombre d'instances de chaque entité : Client 10^7, Product 10^5, OrderLine $4{\cdot}10^9$, Warehouse 200, Stock $2{\cdot}10^7$	Nombre de documents d'une collection ; longueur moyenne des tableaux imbriqués
`distinct_values`	Nombre de valeurs distinctes par clé : IDC, IDP, IDW, brand (5 000), date (365)	Sharding
`array_lengths`	Longueur moyenne imposée pour un tableau : Categories = 2	Tableaux qui ne correspondent pas à une collection

Le nombre de stocks n'est pas donné directement par le sujet. Il se déduit de la règle « même si la quantité est nulle, l'instance existe » : chaque produit a une ligne de stock dans chaque entrepôt, soit $10^5 \times 200 = 2{\cdot}10^7$ stocks.

4. La structure objet

Trois classes très simples, sans logique propre sauf `Database` :

Classe	Attributs	Représente
`Field`	`name`, `kind`, `children`, `item`	Une clé d'un document. `kind` vaut un type primitif (`integer`, `string`, `date`…), `"object"` (sous-clés dans `children`) ou `"array"` (type des éléments dans `item`).
`Collection`	`name`, `fields`, `nb_docs`	Une collection : la liste de ses champs de premier niveau et son nombre de documents.
`Database`	`name`, `collections`, `stats`	Une base complète (DB1…DB5) : dictionnaire nom → Collection, plus les statistiques. Porte toutes les méthodes de calcul.

Un document est donc représenté comme un **arbre** de `Field`. Exemple pour la collection Product de DB2 :

```
Collection "Product"   (nb_docs = 100 000)
├── IDP           integer
├── name          string
├── price         object
│   ├── amount    number
│   ├── currency  string
│   └── vat       integer
├── brand         string
├── description   longstring
├── image_url     string
├── Categories    array — item : object
│                             └── title  string
├── Supplier      object
│   ├── IDS, name, SIRET, headOffice, Revenue
└── Stock         array — item : object
```

```
                                ├── IDW       integer
                                ├── quantity  integer
                                └── location  string
```

Cette représentation arborescente est ce qui permet de traiter **n'importe quelle** dénormalisation avec le même code : DB3 (Product imbriqué dans Stock) ou DB5 (tableau d'OrderLine dans Product) ne sont que des arbres de formes différentes.

5. Le chargement

`parse_field` convertit un nœud du JSON Schema en `Field`. La fonction est **récursive** : pour un objet, elle s'appelle sur chaque propriété ; pour un tableau, sur `items`. Le `format` remplace le type `string` lorsqu'il est présent, ce qui donne directement les types `date` et `longstring`.

```
def parse_field(name, schema):
    kind = schema["type"]
    if kind == "object":
        children = [parse_field(k, v) for k, v in schema["properties"].items()]
        return Field(name, "object", children=children)
    if kind == "array":
        return Field(name, "array", item=parse_field(name, schema["items"]))
    if kind == "string" and "format" in schema:
        return Field(name, schema["format"])  # "date" ou "longstring"
    return Field(name, kind)
```

`load_database` lit les deux fichiers, parcourt les collections à la racine du schéma et récupère le nombre de documents de chacune dans `stats['nb_docs']`. Le nom de la collection sert de clé : en DB3, la collection s'appelle Stock, elle a donc $2{\cdot}10^7$ documents.

```
def load_database(schema_path, stats_path):
    with open(schema_path, encoding="utf-8") as f:
        schema = json.load(f)
    with open(stats_path, encoding="utf-8") as f:
        stats = json.load(f)

    collections = {}
    for coll_name, coll_schema in schema["properties"].items():
        fields = parse_field(coll_name, coll_schema).children
        collections[coll_name] = Collection(coll_name, fields, stats["nb_docs"][coll_name])
    return Database(schema.get("title", schema_path), collections, stats)
```

6. Calcul des tailles (section 2.5)

6.1 Les règles du sujet

Élément	Taille	Constante dans le code
Integer / Number	8 B	`TYPE_SIZES["integer"]`, `["number"]`
String	80 B	`TYPE_SIZES["string"]`
Date	20 B	`TYPE_SIZES["date"]`
LongString	200 B	`TYPE_SIZES["longstring"]`
Clé (paire clé+valeur, tableau)	12 B + valeur	`KEY_SIZE = 12`
1 GB	10^9 B	`GB = 10 ** 9`

6.2 Deux fonctions mutuellement récursives

Le calcul repose sur une distinction simple : la taille d'une **clé** et la taille de sa **valeur**.

- `field_size` = 12 B (la clé) + `value_size` (la valeur) ;
- `value_size` d'un type primitif = la constante du tableau ci-dessus ;
- `value_size` d'un objet = somme des `field_size` de ses sous-clés ;
- `value_size` d'un tableau = longueur moyenne × `value_size` d'un élément.

```
def value_size(self, field, parent_name):
    """Taille de la valeur seule (sans la clé)."""
    if field.kind == "object":
        # Un objet qui correspond à une entité devient le parent de ses tableaux
        if field.name in self.stats["nb_docs"]:
            parent_name = field.name
        return sum(self.field_size(c, parent_name) for c in field.children)
    if field.kind == "array":
        n = self.avg_array_length(field.name, parent_name)
        return n * self.value_size(field.item, parent_name)
    return TYPE_SIZES[field.kind]

def field_size(self, field, parent_name):
    """Taille d'une clé = 12B + taille de sa valeur."""
    return KEY_SIZE + self.value_size(field, parent_name)
```

Un élément de tableau n'a pas de clé (c'est une position), c'est pourquoi on appelle `value_size` et non `field_size` sur `field.item` : les 12 B ne sont comptés qu'une fois, pour la clé du tableau. La taille d'un document est la somme des `field_size` de ses champs de premier niveau (le document lui-même n'a pas de clé) :

```
def document_size(self, coll_name):
    """Taille moyenne d'un document en octets."""
    coll = self.collections[coll_name]
    return sum(self.field_size(f, coll_name) for f in coll.fields)
```

6.3 Longueur moyenne d'un tableau : la notion de parent

C'est le point clé de la logique. Un tableau imbriqué représente une relation 1-n : sa longueur moyenne est le nombre d'instances de l'enfant divisé par le nombre d'instances du parent.

```
def avg_array_length(self, array_name, parent_name):
    """Nombre moyen d'éléments d'un tableau imbriqué.
    Valeur donnée dans les stats (ex. Categories = 2), sinon
    nb(enfant) / nb(parent) (ex. Stock dans Product = 2.10^7 / 10^5 = 200)."""
    if array_name in self.stats["array_lengths"]:
        return self.stats["array_lengths"][array_name]
    nb_docs = self.stats["nb_docs"]
    return nb_docs[array_name] / nb_docs[parent_name]
```

Tableau	Parent	Calcul	Longueur
Categories (toutes DB)	Product	Valeur imposée par `array_lengths`	2
Stock (DB2)	Product	$2{\cdot}10^7 / 10^5$	200
OrderLine (DB5)	Product	$4{\cdot}10^9 / 10^5$	40 000

Le paramètre `parent_name` transporte le nom de l'entité englobante pendant la récursion. Il démarre au nom de la collection et change quand on descend dans un objet qui est lui-même une entité (son nom

figure dans `nb_docs`). Exemple : en DB3, le chemin Stock → Product → Categories fait passer le parent de Stock à Product ; si un tableau d'entité se trouvait dans ce Product imbriqué, son ratio serait bien calculé par rapport à Product.

> **Pourquoi ne pas coder 200 et 40 000 en dur ?** Parce que ces valeurs dépendent du schéma. Avec le ratio, le même code calcule correctement n'importe quelle imbrication, y compris celles du challenge final, à partir des seules statistiques.

6.4 Exemple détaillé : un document Product (DB1)

Clé	Détail	Taille (B)
`IDP`	12 + 8	20
`name`	12 + 80	92
`price`	12 + (amount 20 + currency 92 + vat 20)	144
`brand`	12 + 80	92
`description`	12 + 200	212
`image_url`	12 + 80	92
`Categories`	12 + 2 × (title 12 + 80)	196
`Supplier`	12 + (IDS 20 + name 92 + SIRET 92 + headOffice 92 + Revenue 20)	328
Total		**1 176**

6.5 Effet des imbrications

Base	Document	Calcul	Taille (B)
DB1	Stock	IDP 20 + IDW 20 + quantity 20 + location 92	152
DB1	OrderLine	IDC 20 + IDP 20 + date 32 + quantity 20 + deliveryDate 32 + comment 212 + grade 20	356
DB2	Product	1 176 + clé Stock 12 + 200 × (IDW 20 + quantity 20 + location 92)	27 588
DB3	Stock	Stock sans IDP 132 + clé Product 12 + Product 1 176	1 320
DB4	OrderLine	OrderLine sans IDP 336 + clé Product 12 + Product 1 176	1 524
DB5	Product	1 176 + clé OrderLine 12 + 40 000 × 336	13 441 188

6.6 Collections et base

Une fois la taille d'un document connue, le reste est direct :

```
def collection_size(self, coll_name):
    """Taille d'une collection en octets."""
    return self.document_size(coll_name) * self.collections[coll_name].nb_docs

def database_size(self):
    """Taille de la base en octets."""
    return sum(self.collection_size(name) for name in self.collections)
```

7. Sharding (section 2.6)

7.1 Formules

Avec un sharding sur une clé, tous les documents qui partagent la même valeur de clé sont sur le même serveur. On suppose une répartition uniforme des valeurs sur les serveurs.

- serveurs utilisés = min(nombre de serveurs, nombre de valeurs distinctes de la clé) ;
- documents par serveur = nombre de documents / serveurs utilisés ;
- valeurs distinctes par serveur = nombre de valeurs distinctes / serveurs utilisés.

```
def sharding_stats(self, coll_name, key):
    """Distribution des documents de `coll_name` sur les serveurs avec la clé `key`.
    S'il y a moins de valeurs distinctes que de serveurs, seuls
    `distinct` serveurs reçoivent des données (les autres restent vides)."""
    coll = self.collections[coll_name]
    if find_field(coll.fields, key) is None:
        raise KeyError(f"La clé '{key}' n'existe pas dans {coll_name} ({self.name})")
    servers = self.stats["servers"]
    distinct = self.stats["distinct_values"][key]
    used_servers = min(servers, distinct)
    return {
        "used_servers": used_servers,
        "docs_per_server": coll.nb_docs / used_servers,
        "distinct_per_server": distinct / used_servers,
    }
```

7.2 Le cas « moins de valeurs que de serveurs »

C'est la raison du `min`. Avec `St - #IDW`, il n'existe que 200 entrepôts : une valeur ne peut pas être coupée entre plusieurs serveurs, donc seuls 200 serveurs reçoivent des données (une valeur chacun, 100 000 documents) et 800 serveurs restent vides. Diviser naïvement par 1 000 donnerait 0,2 valeur par serveur, ce qui n'a pas de sens physique.

7.3 Vérification de la clé et clés imbriquées

`find_field` vérifie que la clé de sharding existe dans la collection, en cherchant aussi dans les objets imbriqués : en DB4, `brand` se trouve dans `OrderLine.Product`, et `OL - #brand` fonctionne. La recherche ne descend volontairement pas dans les tableaux : un document contient plusieurs valeurs pour une clé d'un tableau, il ne peut donc pas être rangé sur un seul serveur selon cette clé (MongoDB l'interdit d'ailleurs). Une clé absente lève une `KeyError` explicite.

```
def find_field(fields, name):
    """Cherche une clé, y compris dans les objets imbriqués (pas dans les tableaux :
    on ne peut pas sharder sur une clé contenue dans un tableau)."""
    for f in fields:
        if f.name == name:
            return f
        if f.kind == "object":
            found = find_field(f.children, name)
            if found:
                return found
    return None
```

7.4 Résultats

Stratégie	Docs total	Serveurs utilisés	Docs / serveur	Valeurs distinctes / serveur
St - #IDP	20 000 000	1 000	20 000	100
St - #IDW	20 000 000	200	100 000	1
OL - #IDC	4 000 000 000	1 000	4 000 000	10 000
OL - #IDP	4 000 000 000	1 000	4 000 000	100
Prod - #IDP	100 000	1 000	100	100
Prod - #brand	100 000	1 000	100	5

8. Choix de modélisation et hypothèses

Choix	Justification
Suppression des identifiants redondants dans les imbrications (DB2 `Stock[].IDP`, DB3 `Stock.IDP`, DB4 `OrderLine.IDP`, DB5 `OrderLine[].IDP`)	L'identifiant est déjà porté par le document parent ou par l'objet imbriqué ; le garder dupliquerait l'information et gonflerait la taille ($20\ B \times 4 \cdot 10^9 = 80\ GB$ en DB5).
Les 12 B sont comptés une fois par clé, et une fois pour un tableau (pas par élément)	Lecture directe de « Key+Value pairs/Arrays : 12B + values ». Un élément de tableau n'a pas de nom.
Categories = tableau d'objets `{title}`	Les catégories sont indépendantes les unes des autres (2.3.1) ; un objet permet d'ajouter d'autres attributs de catégorie plus tard.
`price` = objet `{amount, currency, vat}`, `amount` en number	Demandé en 2.3.1 ; un prix est décimal.
$1\ GB = 10^9\ B$	Unité décimale, plus simple pour les calculs à la main.
Longueurs moyennes et répartition uniforme	Le sujet ne fournit que des moyennes ; on ne modélise pas la variance (ex. 1 à 5 catégories).
Pas d'identifiant technique `_id` ni d'index	Hors du périmètre de la section 2.5 ; les index seront traités avec les requêtes (chapitre 3).

9. Résultats et analyse

Base	Signature	Taille (GB)
DB1	Prod{[Cat],Supp}, St, Wa, OL, Cl	1 432,28
DB2	Prod{[Cat],Supp,[St]}, Wa, OL, Cl	1 431,88
DB3	St{Prod{[Cat],Supp}}, Wa, OL, Cl	1 455,52
DB4	St, Wa, OL{Prod{[Cat],Supp}}, Cl	6 104,16
DB5	Prod{[Cat],Supp,[OL]}, St, Wa, Cl	1 352,28

Détail par collection

Base	Collection	Doc (B)	Nb docs	Taille (GB)
DB1	Product	1 176	100 000	0,1176
DB1	Stock	152	20 000 000	3,0400
DB1	Warehouse	132	200	0,0000
DB1	OrderLine	356	4 000 000 000	1 424,0000
DB1	Client	512	10 000 000	5,1200
DB2	Product	27 588	100 000	2,7588
DB2	Warehouse	132	200	0,0000
DB2	OrderLine	356	4 000 000 000	1 424,0000
DB2	Client	512	10 000 000	5,1200
DB3	Stock	1 320	20 000 000	26,4000
DB3	Warehouse	132	200	0,0000
DB3	OrderLine	356	4 000 000 000	1 424,0000
DB3	Client	512	10 000 000	5,1200
DB4	Stock	152	20 000 000	3,0400
DB4	Warehouse	132	200	0,0000
DB4	OrderLine	1 524	4 000 000 000	6 096,0000
DB4	Client	512	10 000 000	5,1200
DB5	Product	13 441 188	100 000	1 344,1188
DB5	Stock	152	20 000 000	3,0400
DB5	Warehouse	132	200	0,0000
DB5	Client	512	10 000 000	5,1200

Lecture des résultats (lien avec 2.5.4)

- **OrderLine domine tout** : $4 \cdot 10^9$ documents, environ 1 424 GB sur ~1 432 GB en DB1. Toute décision qui touche OrderLine a un impact massif.
- **DB4 est la plus volumineuse (×4,3)** : les 1 176 B de Product sont recopiés dans chacune des $4 \cdot 10^9$ lignes de commande. Un changement de prix doit être répercuté sur ~40 000 documents par produit (anomalies de mise à jour).
- **DB3** recopie Product dans ses 200 stocks : +23 GB, et les mêmes problèmes de mise à jour à plus petite échelle.
- **DB2 et DB5 sont légèrement plus petites que DB1** : un tableau imbriqué ne duplique rien (relation 1-n vue du côté 1), et on économise l'IDP retiré de chaque élément.
- **DB5 crée des documents énormes** : ~13,4 MB par produit, proche de la limite de 16 MB d'un document MongoDB, et le tableau grossit à chaque nouvelle commande. Lire un produit implique de charger toutes ses commandes.

10. Tests

Les tests (`python -m unittest`) comparent le code à des valeurs calculées à la main, indépendamment du code :

Test	Ce qui est vérifié
`test_document_sizes_db1`	Taille des 5 documents de DB1 (types primitifs, objets, petit tableau).
`test_nested_arrays`	Longueur des tableaux par ratio : 200 stocks (DB2), 40 000 lignes (DB5).
`test_nested_objects`	Objet Product imbriqué dans Stock (DB3) et OrderLine (DB4).
`test_database_size_db1`	Somme des collections.
`test_more_values_than_servers`	`St - #IDP` : 1 000 serveurs, 20 000 docs, 100 valeurs.
`test_fewer_values_than_servers`	`St - #IDW` : seulement 200 serveurs utilisés.
`test_nested_key`	Sharding sur `brand` imbriqué dans OrderLine.Product (DB4).
`test_unknown_key`	Une clé absente de la collection lève une erreur.

11. Utilisation et suite du projet

```
from bigdata_sim import load_database

db = load_database("schema/db2.json", "stats.json")
db.document_size("Product")             # 27588 (octets)
db.collection_size("Product")           # 2758800000 (octets)
db.database_size()                      # taille totale (octets)
db.sharding_stats("Product", "brand")   # {'used_servers': 1000, 'docs_per_server': 100.0, ...}
```

Pour les TD suivants, les opérateurs (filtre, nested loop, agrégat) recevront un objet `Database` et le nom d'une collection. Ils disposeront directement de la taille d'un document, du nombre de documents, des valeurs distinctes (pour la sélectivité) et de la répartition par serveur : tout ce qu'il faut pour estimer les volumes échangés et les coûts.

12. Questions probables à l'oral

Pourquoi 12 B ne sont-ils pas comptés pour chaque élément d'un tableau ?

> Un élément de tableau n'a pas de clé, seulement une position. Les 12 B correspondent à la clé du tableau ; chaque élément objet paie ensuite 12 B par sous-clé.

D'où vient la longueur 200 du tableau Stock en DB2 ?

> Du ratio nb_docs(Stock) / nb_docs(Product) = $2 \cdot 10^7 / 10^5$, calculé par `avg_array_length`.

Pourquoi DB2 est-elle plus petite que DB1 alors qu'on a dénormalisé ?

> Imbriquer du côté « 1 » d'une relation 1-n ne duplique aucune donnée ; on retire en plus l'IDP de chaque stock (20 B × $2 \cdot 10^7$ = 0,4 GB).

Que se passe-t-il pour St - #IDW ?

> 200 valeurs pour 1 000 serveurs : seuls 200 serveurs sont utilisés, avec 100 000 documents chacun ; 800 serveurs sont vides. D'où le `min` dans `sharding_stats`.

Comment le programme sait-il qu'un champ est une date ou un texte long ?

Par le mot-clé `format` du JSON Schema, lu dans `parse_field`.

Que faut-il changer pour un nouveau schéma ?

Uniquement les fichiers JSON : le schéma et `stats.json` (nb_docs des nouvelles entités, valeurs distinctes des clés de sharding, longueurs imposées éventuelles).